# Cloud design studio

CPSC 436C 2026W1

**How to work.** In pairs or groups of three. Make a copy of this document and write in your copy. No laptop? Write on the printed copy and photograph it.

**Submitting, 2%.**

| When | What to do | Worth | Join |
|---|---|---|---|
| During class | Join Formative at **https://frm.tv/join/W5G5TT** (or scan the code) and submit at each 📤 checkpoint below.<br><br>**Note: you can just use your cwl instead of names in Formative (but do add names on canvas)** | 1% | |
| Before Thursday's lecture (8 October 2026) | Export your finished copy as a PDF and upload it to the Canvas group assignment. It is marked on completeness, using the checklist at the end of this sheet. | 1% | |

Rough numbers are fine everywhere, prices included: say where each one comes from. If two of your estimates disagree, keep both and say which one you would measure first. Add rows or lines wherever you need them. Everything runs in AWS's Canada (Central) region, ca-central-1. You may use services that are not on the service cards; say what each one does.

**Names:**

## Part 1: a small pipeline

💼 Business case

A mid-sized city's transit agency runs about 400 buses on 80 routes, from 05:00 until 01:00 the next night. Every bus has an on-board unit that records its position every 30 seconds. Today those records are thrown away at the end of each day.

That has become a problem. Complaints about late and skipped buses have grown this year, and customer service cannot check them. The planning team is starting a review of its routes and wants to see how long each route really takes. Finance has approved a small pilot, and the agency's IT team, two people who look after the website and the office network, has asked you to design it: a pipeline in the agency's AWS account that keeps the buses' records and lets these people answer their questions.

**The data.**

- Each bus can upload one file per trip when it reaches the trip's final stop: JSON, one record per 30 seconds, about 150 bytes each, with the bus's id, its trip, its route, its position (latitude and longitude), its next stop and a timestamp. The upload can go to an S3 bucket the pipeline provides; IT gives each on-board unit a credential that can only upload. The unit names each file by service day, bus and trip, in that order (`2026-10-04_bus-117_trip-09.json`), and you choose the folders in front of that name.
- A bus makes about 20 trips a day, each about an hour long, so a trip file holds about 120 records.
- At a few stops there is no signal. A bus that cannot upload keeps its files and sends them all when it returns to the depot, between 01:00 and 01:30.
- By 01:30 each night, operations has a short list of how many trips each bus ran that day.
- Stored as Parquet, records like these take about a tenth of the space of the JSON.
- Below, *a day* means one service day, from 05:00 to 01:00, and *yesterday* means the last one.

**Who needs it, and what for.**

| Who | What they need | How often, how much |
|---|---|---|
| Customer service | Where a given bus was, minute by minute, on the day a rider complained | A few complaints a day, about earlier days up to 2 years back; an answer within 10 minutes, in a web browser (they do not write code) |
| Two route planners | How long each route takes, compared month by month; they work in Python notebooks | Most afternoons, each query on one route over 3 to 6 months, from notebooks on their own laptops |
| The operations director | Yesterday's records complete by 06:00 each morning | Every day; at most 1% of yesterday's trips may be missing |
| Finance | A pilot that costs little | About $20 a month for the first year; queries should cost nothing while nobody asks |

The agency keeps the records for 2 years. Whether to keep the uploaded files as they arrived, and for how long, is yours to propose. **Out of scope for now:** riders' apps, timetables, anything about passengers.

Sam, the agency's only data engineer, started on this pilot before retiring early last month. Sam's notes are below, unfinished: use them as a starting point, not as answers.

⚙️ Data engineering notes, started by Sam before an early retirement (unfinished)

- The trip files are small and many: about 8,000 a day. Somebody has to put them together, or every reader pays for it.
- Customer service asks by bus and day; the planners ask by route over months. One layout will not serve both equally well. Decide who comes first, and why.
- Keep the uploaded files as they arrived? Without them nothing can be re-run after a bug. Storage is cheap, but how cheap after two years?
- Some buses only upload at the depot after 01:00. What does “complete by 06:00” mean for them?
- Check what bills when nobody asks. Finance will.
- Things to look up before promising anything: …

### Task 1: requirements

Write what the pipeline must do, one requirement per line. To each requirement add its constraints: how fast, how fresh, how complete, how much it may cost, with a number. For example, *“the dashboard is updated every 10 minutes”* is a constraint on a requirement to show a dashboard.

One way to write them so that each can be checked is the EARS form. Two examples:

- *WHEN customer service asks for one vehicle on one earlier day THE PIPELINE SHALL return its position for every minute of that day, within 10 minutes.*
- *IF a trip file arrives after its day is complete THEN THE PIPELINE SHALL add it to that day.*

| | Requirement | Its constraints |
|---|---|---|
| R1 | | |
| R2 | | |
| R3 | | |
| R4 | | |
| R5 | | |

📤 **Checkpoint 1.** Paste your requirements and their constraints into Formative.

### Task 2: estimates

| | Your estimate | How you got it |
|---|---|---|
| Trip files and records in a day | | |
| Raw JSON a day, and as Parquet a day | | |
| Cost, $ a month | | |

### Task 3: the design

⚙️ Data engineering lens

Many small files arriving all day and questions that are rare: for each step, decide what starts it, where its bytes land and how they are laid out, and what it costs a month.

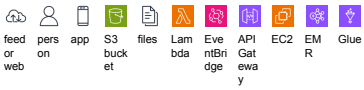
Design the smallest pipeline that meets your requirements. Write one row for each step:

| Step | What starts it | What runs it | Where its output lives (bucket, folder layout) | $ a month | Requirements |
|---|---|---|---|---|---|
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |

*Sketch:*

*(you can insert drawing, screenshot, or any other type of diagram)*

Icons to copy into your sketch:




📤 **Checkpoint 2.** Upload your sketch to formative and copy-paste table

## Part 2: across the province

💼 Business case

A year later the pilot works, and it has been noticed. The provincial transit authority is taking your pipeline over for all 15 transit agencies in the province. The authority's data team is four people.

**The data.**

- About 4,000 vehicles on about 1,000 routes, all uploading trip files in the same format: about 80,000 files a day.
- The authority keeps two years of records. After its first year at this size the store holds about 3.5 billion records, about 45 GB of Parquet, plus the uploaded files you chose to keep.
- The agencies' timetables are now in scope: one GTFS file of about 50 MB per agency, uploaded by the agency to a bucket you choose and changed every few weeks. Every version is kept, so each past day can be compared with the timetable in force on that day.
- Each record's trip id is the trip id in its agency's timetable.
- A vehicle's delay at a stop is the time its next stop changed away from that stop, minus the time the timetable gives for that stop.
- Each agency also publishes a live feed (its GTFS-Realtime feed): one HTTP request returns where its vehicles are right now, about 150 bytes per vehicle, updated every 30 seconds. The feed keeps no history.

**Who needs it, and what for.** The **bold** need is required; choose one or two more to design for.

| | What they need now | How often, how much |
|---|---|---|
| ach agency's anagement | A dashboard each morning: the average delay of every route for every hour of yesterday, comparing each record with the timetable | Every day, ready by 07:00 |
| oute anners | Every trip's start, end and delay, by route over months, in their notebooks | Their queries now take minutes; they want seconds |
| oute anners | Whenever the definition of delay changes, every past trip recomputed overnight | About once a month. A planner ran it over one year of records, read from the Parqu files in S3, with DuckDB on all cores of the largest EC2 instance type the authority's allows (16 vCPUs, 64 GB of memory; clusters and functions have no such limit): ab hours. It must run between 01:30 and 05:00, alongside the nightly jobs |
| **data cientist** | **An arrival-time model: a regression retrained every week on a training table with one row for every record of the past year, with the average delay at that record's stop, for that hour of the week, over the 8 whole weeks before the record's week** | **The table and the retrained model are ready every Monday by 06:00. The table larger than the memory of the largest instance IT allows** |
| iders, rough the uthority's new pp | When the next vehicle reaches their stop, from the model and where the vehicles are right now | About 1 million requests a day: up to 50 a second at rush hour, close to none at nig each answer within a second |
| inance | A budget that grows with what the system does | About $1,500 a month |

⚙️ Sam's notes, continued (also unfinished)

- Fifteen agencies, one format: does anything in Part 1 break at ten times the files?
- The training table looks back 8 weeks from every record. That is not something one day's files can answer on their own.
- Riders want *now*; everyone else wants *yesterday* or *the last two years*. Are these the same pipeline?
- The 9 hours was measured on one instance with every core busy. The window is three and a half hours.
- Finance's number is a ceiling, not a target.

## Task 4: requirements and design, scaled up

⚙️ Data engineering lens

Some of the new work outgrows one instance and some must answer within a second: for each need, decide where it runs from how much it reads and how fast it must finish, how its output is laid out for the people who read it, and what that costs.

Write a row for each step of Part 1 that changes, then one for the required need (the training table and weekly model), then one for each need you chose. Start each row with *New* or *Changed*. For a job that runs less often than daily, give its cost spread over a month.

| Requirement and its constraints | What it reads, and how much | What runs it, and why that and not something smaller (or write *nothing smaller*) | Where its output lives (bucket, folder layout) | $ a month |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |

📤 **Checkpoint 3.** In Formative add your updated requirements, 2-3 sentence summary

*Sketch of the changed design:*

Icons to copy into your sketch:




📤 **Checkpoint 4.** Upload your sketch to Formative

**Tools** that run on an instance or in a function: pandas, DuckDB, JupyterLab, PySpark.

## Guiding questions

Use these to check your design.

| | Question | Read |
|---|---|---|
| Data and readers | Who reads the result, and is it operational (answers someone now) or analytical (looks back over time)? | L1–L2 |
| | Which questions will people ask, how often, and over how much data? | FDE ch. 6 |
| | How does new data arrive: appended, in batches, or updated in place? | FDE ch. 6 |
| Load and cost | Is the load steady, bursty or rare, and what does the system cost when nothing happens? | FDE ch. 4 |
| | Does it fit on one instance, or does it outgrow one? | L4–L5 |
| | What is the total cost: running it, the people who run it, and what leaving costs if it turns out wrong? | FDE ch. 4 |
| Storage | Where does the data live after each step, and does it outlive the compute that made it? | FDE ch. 6 |
| | Do you keep the raw data, for how long, and who decides? | FDE ch. 6 |
| Team | How much of this does your team want to run itself? | FDE ch. 4 |

| Question | Read |
| --- | --- |
| How do you design analytics workloads to withstand and mitigate failures? | |

## Before Thursday: completeness checklist

The Canvas upload earns its 1% when all of these are there:

- ☐ Names of everyone in the group
- ☐ Task 1: each requirement has at least one constraint with a number
- ☐ Task 2: all three estimates, each with how you got it
- ☐ Task 3: each step says what starts it, what runs it, where its output lives and its $ a month; a sketch
- ☐ Task 4: a row for each changed step, one for the required need, and one for each need you chose; a sketch